Day 2 — Mappings, service levels, the platform and Docker

Data Creators › Day 2 — anchor notebook

© 2026 Decoded Limited. All rights reserved. decoded.com

About this notebook

- Day 2's anchor: the agenda, then one numbered section per topic. Each section runs as a cycle: a demo, an activity or challenge, a shareback, a break.
- Every concept has its diagram; every demo has the cells that run it; every activity has its brief; every challenge points into the training repository.
- Run the **Setup** cell first. Cells that start with `!` run in a shell in this notebook's folder with the demo environment on `PATH`.
- 🖉 Whiteboard lines are the questions the room works through together.
- The last section, **Questions from the room**, collects the day's questions and their answers as they come.

```
# Setup: the demo environment on PATH, DuckDB and pandas for showing results, a quick diagram helper
import os, sys, subprocess, textwrap
from pathlib import Path
from IPython.display import Image, Markdown, display

VENV = Path(os.environ.get("CG_DXP_VENV", Path.home() / ".venvs" / "cg-dxp"))
os.environ["PATH"] = ":".join([str(VENV / "bin"), str(Path.home() / ".local" / "bin"),
                               "/Applications/Docker.app/Contents/Resources/bin", "/opt/homebrew/bin", os.environ["PATH"]])
os.environ.setdefault("DBT_SEND_ANONYMOUS_USAGE_STATS", "false")
HERE = Path.cwd()                       # courseware/day2-*/
COMMON = HERE.parent / "common"
assert COMMON.exists(), "Open this notebook from its own folder (courseware/day2-*)"

def quick_diagram(mermaid: str, name: str = "scratch"):
    """Render a Mermaid diagram typed live and show it inline (needs node; first use downloads mermaid-cli)."""
    src = HERE / f"{name}.mmd"; out = HERE / f"{name}.png"
    src.write_text(textwrap.dedent(mermaid).strip() + "\n")
    cfg = COMMON / "diagrams" / "mermaid-config.json"
    r = subprocess.run(["npx", "-y", "-p", "@mermaid-js/mermaid-cli@11.17.0", "mmdc", "-i", str(src), "-o", str(out), "-c", str(cfg), "-b", "white", "-s", "2"],
                       capture_output=True, text=True)
    if r.returncode != 0:
        print(r.stderr[-800:]); return
    display(Image(filename=str(out)))

def sql(query: str, db: Path, read_only: bool = True):
    """Run a query against a DuckDB file and show it as a table."""
    import duckdb, pandas as pd
    pd.set_option("display.width", 200); pd.set_option("display.max_columns", 30)
    con = duckdb.connect(str(db), read_only=read_only)
    try:
        return con.execute(query).df()
    finally:
        con.close()

print("✅ Day 2 setup:", subprocess.run(["dbt", "--version"], capture_output=True, text=True).stdout.split("\n")[1].strip() if subprocess.run(["which", "dbt"], capture_output=True).returncode == 0 else "dbt not on PATH: run common/env/setup-env.sh")
```

Today

Yesterday's contract is today's input. By the end of the day it has mapping rules and measurable commitments, you know what the platform does with your tables, and you have built and run a container. The code comes tomorrow.

#	Section	What happens
1	**Day 1 recap and the map of the week**	Recap: what we made yesterday · Whiteboard: source-, domain- and consumer-aligned · Today, and the two musts
2	**Mappings and service commitments**	Demo: STTM and service levels · Worked example: the running example's contract, mapping and service levels · Activity 6: extend the contract · Break · Shareback: review and present a contract
3	**Platform architecture**	Demo: warehouse, lake, lakehouse; Databricks under the hood; Delta; the tools together · Shareback
4	**Docker**	Demo: Docker in the six steps of the challenge, a sample Dockerfile, the gate · Demo: the dev container's Dockerfile · Demo (optional): two containers with Compose, and the network between them · Challenge 7: Docker practical · Lunch · Challenge 7 continued, with the stretch · Shareback: image, container, what survives
5	**Git and GitHub (optional)**	Demo: the loop on a scratch repository, then the pull request on screen · Optional practical: branch, commit, push, pull request · Break
6	**Catalyst skills, and setting up for tomorrow**	Demo: the skills, their tiers and the lifecycle; how to load them · Activity 8 (if time allows): one design skill on your product · Databricks setup · Close

1. Day 1 recap and the map of the week

Recap — what we made yesterday

WHAT, WHY AND HOW

What did we make yesterday, and where does it go today?

WHAT
A list of the room's products and pain points; PRDs and data contracts generated from interviews and reviewed; the vocabulary of the lifecycle.

WHY
Today's design morning extends those contracts; the practicals from this afternoon on exist to check what the design produces.

HOW
Ask the room for each answer and write it up; add what is missing; then the alignment picture; then the agenda.

Whiteboard — Day 1 recap

- What is a data product?
- What did we learn about finding out what to build?
- What did the generated documents get wrong?
- Which documents exist now, and in what order?
- What is the week's thread?

Whiteboard — source-, domain- and consumer-aligned

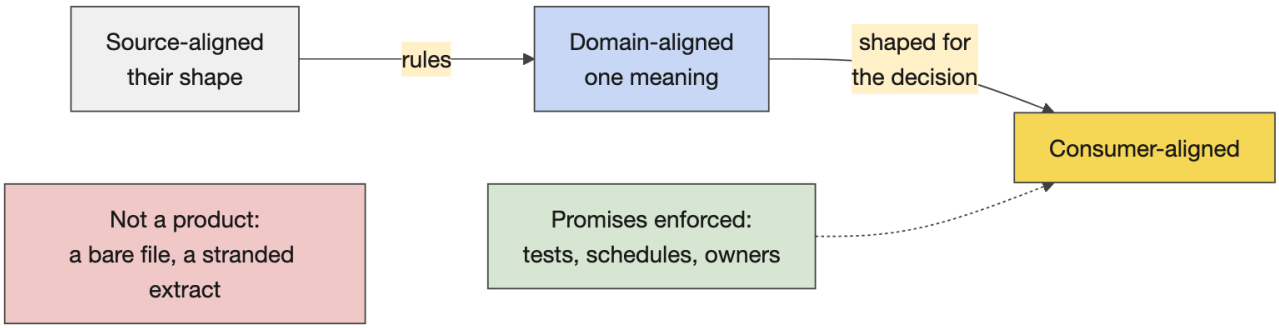
WHAT, WHY AND HOW

Yesterday's contract describes a table. Whose shape is that table?

WHAT
The consumer's. The sources keep the producer's shape; the mapping you write today is the path between; the firm's agreed definitions sit in the middle.

WHY
The three words are how CG names layers, catalogs and products; every model you build this week belongs to one of them.

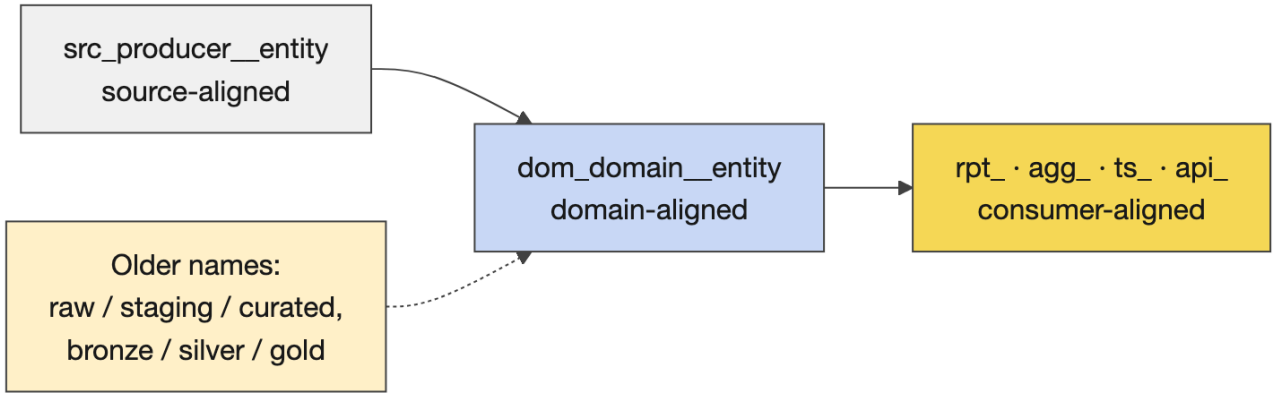
HOW
Three one-liners, the running example, and the older names you will hear.



- Source-aligned: the producer's shape, kept faithfully, as an audit path back to the extract.
- Domain-aligned: the firm's one definition of each entity, agreed by the data owner, reused by every consumer.
- Consumer-aligned: the shape of the decision, one row per business question.

🖉 **Whiteboard — source-, domain- and consumer-aligned**

- What is source-aligned data?
- What is consumer-aligned data?
- So what is domain-aligned?
- Where does today's mapping fit?
- What about staging, intermediate, marts, or bronze, silver, gold?



- src_, dom_, and rpt_/agg_/ts_/api_ are the prefixes Friday's project uses.
- Recognise raw, staging, intermediate, curated and bronze, silver, gold when you meet them; speak alignment.

Today — and the two musts

Two things must happen today, whatever else slips: the **design documents before the first break** (the mapping and the service levels into your contract), and **Docker done by the end of the day** (Challenge 7 at PASS). The platform block in between is there so the practical makes sense; the afternoon's skills overview is there so tomorrow's code comes from the right place. No dbt code is generated today; the design is agreed first.

2. Mappings and service commitments

Demo — STTM and service levels

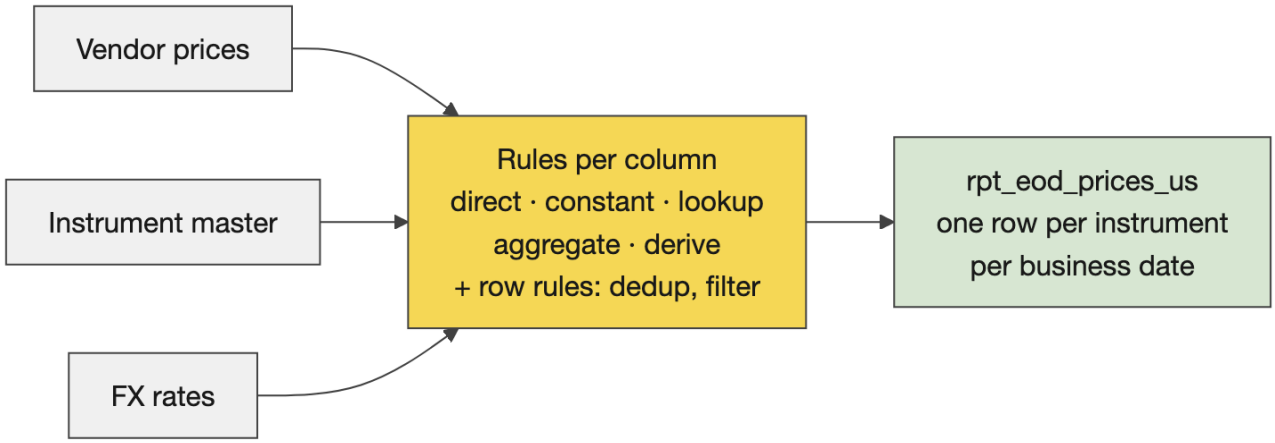
WHAT, WHY AND HOW

What is the core of a source-to-target mapping?

WHAT
The transformation rules: one per target column, plus the row rules, in English precise enough to implement.

WHY
The STTM is where the product's business logic lives; a vague rule becomes a wrong join nobody notices.

HOW
Name the rule type for every column, then the edge cases: nulls, zeros, duplicates, dates, time zones.



- Direct, constant, lookup, aggregate, derive: the textbook five; Catalyst splits derive further.
- Row rules matter as much as column rules: dedup, filter, reject, and what a null means in each column.
- The running example's mapping is in common/running-example/04-sttm.md: eleven column rules and four row rules.

How a rule is written

One row per target column. The columns of the mapping, and what goes in each:

Column	What it holds
Target column	The name in the consumer-aligned table, from the contract's schema
Type	The target data type, with precision for money and rates (`DECIMAL(18,6)`)
Rule type	One of the five below: it tells a reviewer what kind of work the column needs before reading the rule
Source	Which source table and column, by alias (V for the vendor feed, I for the instrument master, F for FX)
Rule, in English	Precise enough to implement without asking: the operation, the join key, what null means, zero and negative, which value wins

Row rules sit beneath the column rules and say which rows exist at all: dedup (which delivery wins), filter (which rows are in scope), reject (what cannot be placed and is dropped), and what a null means per column.

The five rule types

Rule type	What it means	Running example	What a reviewer checks
Direct	Copy the source value, perhaps renamed, trimmed or cast	`price_date` from `V.px_date`, cast to date: the business date of the close, not the delivery time	The type and the cast; what null or an unexpected value means
Constant	A fixed value with no source column	`price_source = 'VENDOR'` (`'MANUAL'` reserved for an override that does not exist yet)	That the value list is closed and documented
Lookup	A value fetched from another table by a key	`ticker` from the instrument master, joined on `upper(trim(isin))`; a missing master row leaves it null and fails the relationship test	That the key is unique on the lookup side; what happens with zero or several matches
Aggregate	One value from several source rows	The latest delivery per instrument and date (row rule R1: the row with the latest `vendor_ts` wins)	The grain, and the tie-break
Derive	Compute from one or more inputs with an operation	`price_usd = price_native * fx_rate_to_usd`, null when the rate is null so the build fails; `price_key = instrument_id || '|' || price_date`	The units, the null arithmetic, and the test that guards the result

Catalyst's `transformation-spec` returns a CG-specific list that splits derive further (conversions, concatenations, conditional logic); the five above are the textbook set and every CG type is one of them.

🖉 Whiteboard — STTM

- What is a source-to-target mapping? What is its core?
- What kinds of rule can a column have?
- What is in the design documentation set?

WHAT, WHY AND HOW

Delay is allowed to be an hour. Is that the indicator, the objective or the agreement?

WHAT

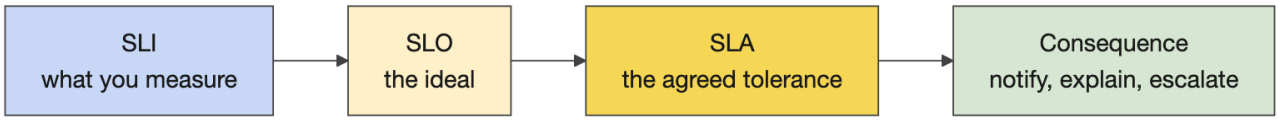
SLI: the measure. SLO: its ideal value. SLA: the permissible departure, agreed, with a consequence.

WHY

Consumers hold you to commitments, not to good intentions; each commitment has a cost.

HOW

List the indicators a product needs, give each a target and a tolerance, and say how it is measured.



- The indicator is a number you can measure every run; the objective is zero delay, 100% complete; the agreement is what you actually sign.
- Delivery time needs a time zone, the eligible days, the measurement point and the action on failure.
- Duty to explain: when a number looks wrong, somebody owes the consumer an answer within an agreed time.

Whiteboard — service levels

- What do SLI, SLO and SLA stand for, and what do they mean?
- Give me indicators.
- What is availability, and what does 99.9% mean?
- How do providers get more nines?

Availability	Downtime a year	Remember it as
90%	about 36 days	a month
99%	about 3.65 days	a working week
99.9%	about 8.8 hours	a working day
99.99%	about 53 minutes	an hour
99.999%	about 5 minutes	a coffee

Worked example — the running example's contract, mapping and service levels

> **DEMO — a finished set, before you extend your own**
>
> Screen-share the running example: the contract, then its mapping and service levels as separate documents.

1. Open `common/running-example/03-data-contract.md` in the Markdown preview: section 2 (the schema, the target), section 3 (the SLAs) and section 4 (the quality rules).
2. Open `04-sttm.md`: one rule per target column with its type, rule type, source and the rule in English; the four row rules beneath; the edge cases agreed in the interview.
3. Open `05-service-levels.md`: each commitment as an indicator, an objective and an agreement, with its clock and how it is measured.
4. Open `data/`: the three source files as landed, the intermediate table and the price sheet. The contract's schema is the price sheet's, not the vendor's; the last cell shows the first rows of each.

```
from IPython.display import Markdown, display
display(Markdown((COMMON / "running-example" / "04-sttm.md").read_text()))
```

```
display(Markdown((COMMON / "running-example" / "05-service-levels.md").read_text()))
```

```
import pandas as pd
D = COMMON / "running-example" / "data"
for name in ["vendor_prices_raw", "instrument_master_raw", "fx_rates_raw"]:
    print(f"--- {name}.csv: a source, as landed"); display(pd.read_csv(D / f"
{name}.csv").head(4))
print("--- rpt_eod_prices_us.csv: the target, the schema in contract section 2");
display(pd.read_csv(D / "rpt_eod_prices_us.csv").head(6))
# The sources keep the vendor's names (isin, px_close, gbp); the target has the
consumer's (instrument_id, price_usd, GBP). data/README.md says which rows show each
mapping rule.
```

Activity 6 — extend the contract

🎯 ACTIVITY 6 — Extend the contract · Individual, then pairs or tables · 30 min

On your own, ask Catalyst what source-to-target transformation types and what service-level indicators and objectives it knows. Then, in pairs or at your table, take one of yesterday's data contracts: name the incoming tables and the outgoing table, write the rules that get you from one to the other (lookups, derivations, filters, what nulls mean), and the commitments a consumer would hold you to. Transcribe the discussion, give it to Catalyst, extend the contract, then inspect and revise.

Produce:
- At least one unambiguous mapping rule someone else could implement without asking you
- At least one measurable commitment with its indicator, target, tolerance and time zone
- The revised contract, shared for review

Finished early? Add the breach response: who is told, by when, what they are told.

Debrief questions:
- Could someone else implement this rule without asking you what it means?
- For 'available by 07:30': which time zone, which days, measured where, and what happens on failure?
- For a lookup: what is the join key, and what happens with zero or several matches?

🤖 PROMPT

LEARNERS Copilot Chat · Chat view

```
What types of source-to-target transformation do you distinguish, and
what service-level indicators and objectives should a data contract
state?
```

/ai-catalyst data-contract Extend the attached contract with an STTM section (one rule per target column, plus row rules) and measurable SLAs from this discussion transcript. Keep every other section as it is.· INSTRUCTOR Claude Code

Same questions, then: /data-contract Extend ... (attach common/running-example/03-data-contract.md and the discussion)

☕ **BREAK · 15 min · back at 10:30**

Questions go in the table at the end; the next part starts on time.

Shareback — review and present a contract

🖥️ **DEMO — Markdown editor, then a self-contained HTML page**

Read Markdown properly, then turn the contract into one HTML page on a small model and on a big one.

1. Right-click a submitted contract, Open with..., Markdown editor (or Open Preview); edit one cell.
2. In Claude Code (or Copilot) run the prompt below on a fast model at low effort.
3. Open the result in the browser: contents, overview, diagrams.
4. Start the same prompt on the biggest model at high effort; let it run while you teach; compare when it finishes.

🤖 **PROMPT**

LEARNERS Copilot Chat · Chat view, contract attached

```
Create a well-structured, self-contained HTML document from this
contract: an overview at the top, a table of contents, diagrams where
they help (sources to target, the SLA timeline), and plain-English
prose explaining any section a non-engineer would find hard. One file,
no external dependencies.
```

INSTRUCTOR Claude Code

```
Same prompt. Fallback already rendered: common/running-example/03-data-
contract.html
```

```python
# Fallback: the contract already rendered as HTML (made with pandoc; the AI version
adds prose and diagrams)
import webbrowser
webbrowser.open((COMMON / "running-example" / "03-data-contract.html").as_uri())
print("opened", COMMON / "running-example" / "03-data-contract.html")
```

Could someone else implement this rule without asking you what it means?

Take two rules from the room's contracts and test them against this list.

Ambiguity	Ask	Running-example answer
Identifiers and keys	Which column joins, and is it unique on the lookup side?	ISIN, one primary row per instrument in the master
Null	What does empty mean in this column?	A null FX rate means no same-day rate; the USD price is null and the build fails
Zero, minimum, maximum	Is zero a value or a missing value? What is implausible?	A zero or negative price is a quality failure
Duplicates	Can the source send the same row twice? Which wins?	Re-deliveries: latest vendor timestamp wins
Dates and time zones	Business date or delivery time? Which zone?	Business date of the close; 07:30 America/New_York
Scope words	'US', 'active', 'price': whose definition?	Market and active flag from the master; price is the primary-listing close

3. Platform architecture

Demo — warehouse, lake and lakehouse; Databricks under the hood; Delta; the tools together

WHAT, WHY AND HOW

Why is Databricks on the paved path, and what does it actually do with your dbt SQL?

WHAT

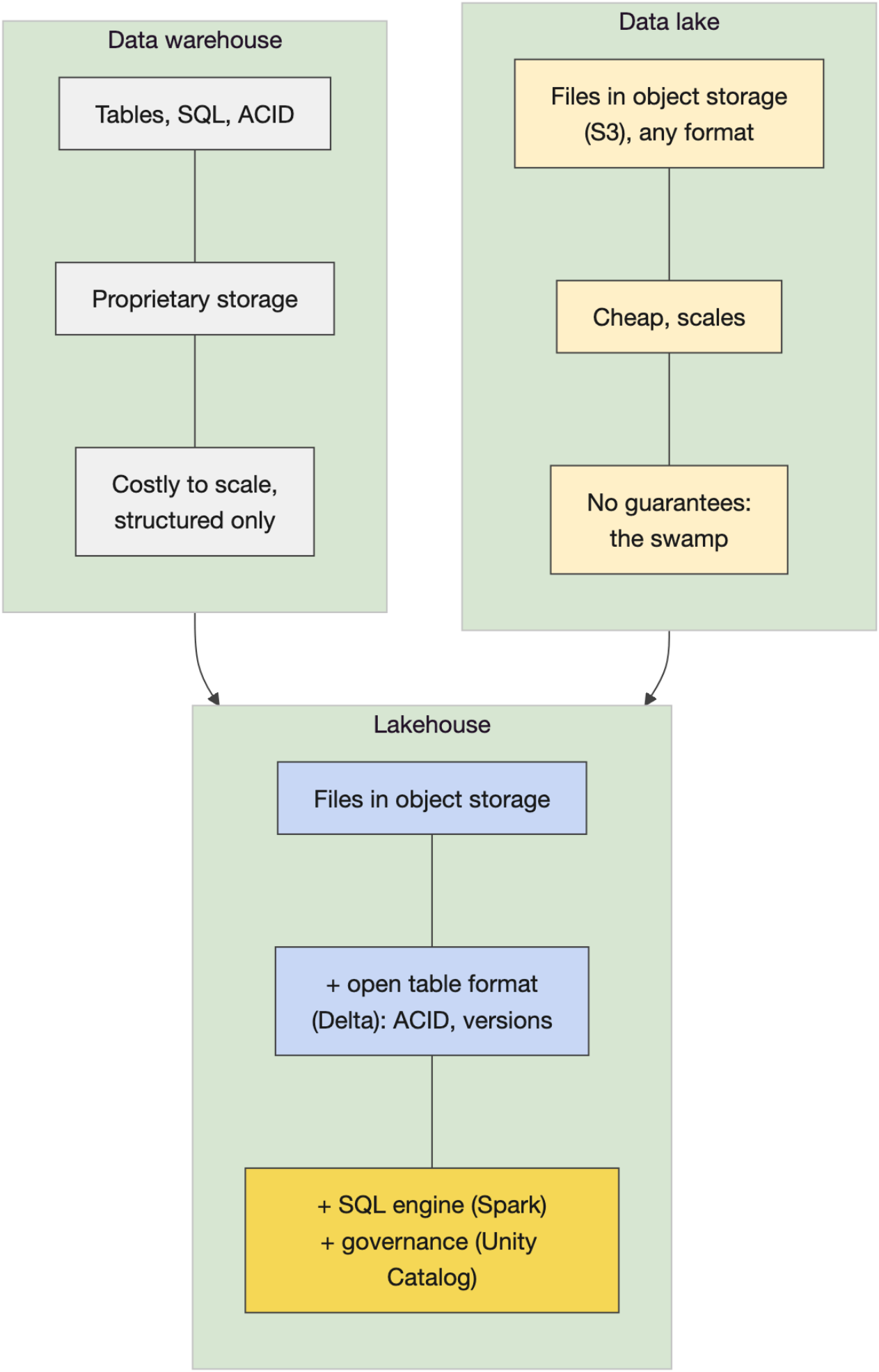
A lakehouse: Delta tables in S3 (files plus a transaction log), Spark compute run by Databricks, Unity Catalog on top.

WHY

Every dbt model you build runs here; knowing what happens under the hood explains the costs, the guarantees and the limits.

HOW

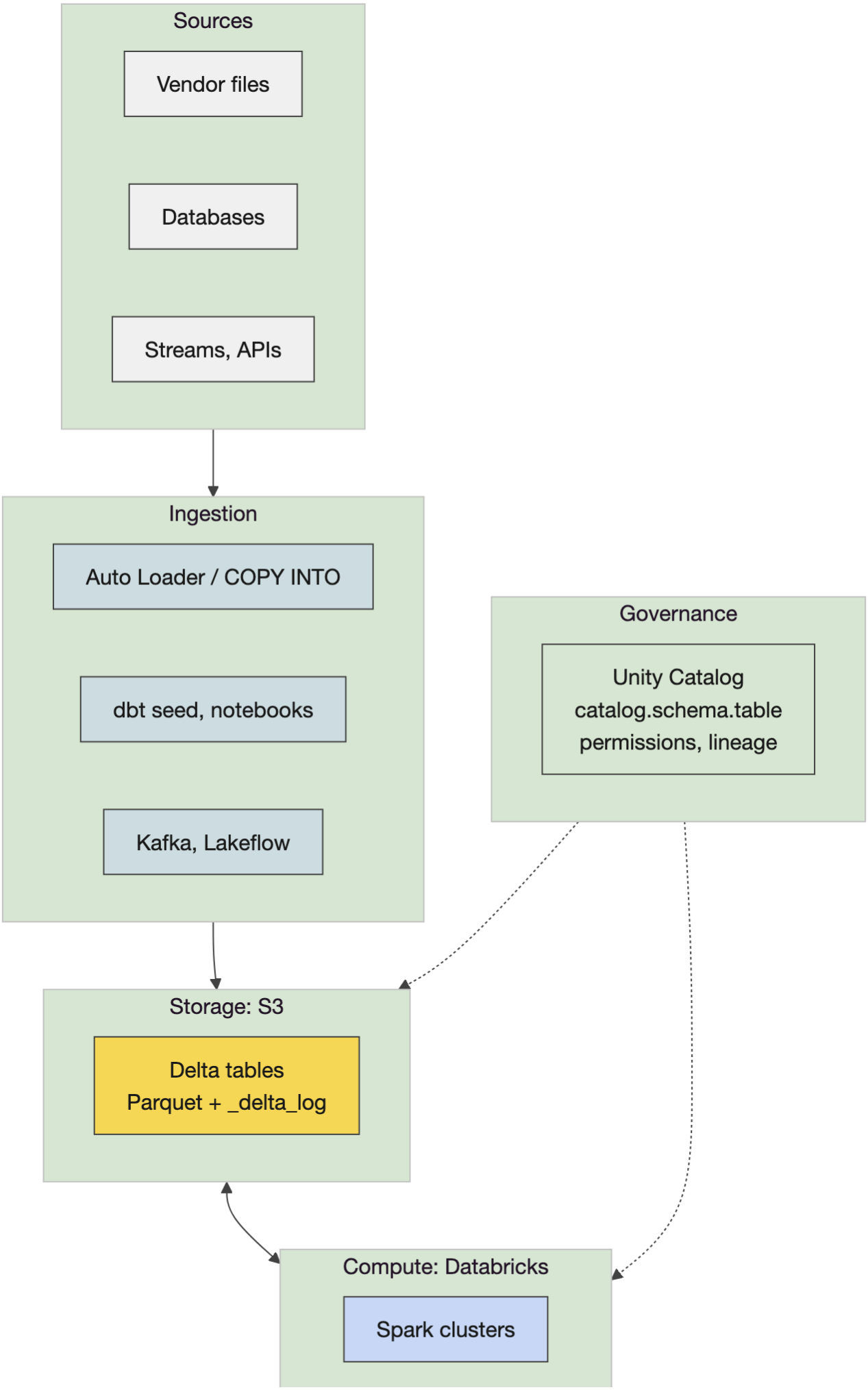
Three pictures (warehouse, lake, lakehouse; Databricks under the hood; the Delta format), the Delta demo on this laptop, then how all the tools fit together. Spark only if ahead.

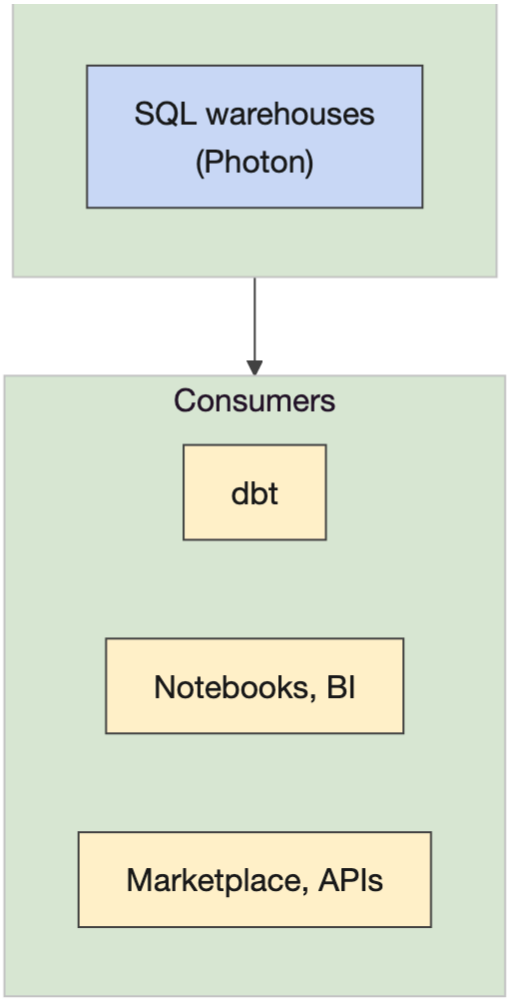

Data warehouse
Tables, SQL, ACID
Proprietary storage
Costly to scale, structured only
Data lake
Files in object storage (S3), any format
Cheap, scales
No guarantees: the swamp
Lakehouse
Files in object storage
+ open table format (Delta): ACID, versions
+ SQL engine (Spark) + governance (Unity Catalog)

- A warehouse gives tables, SQL and ACID on its own storage: strong guarantees, expensive to scale, structured data only.
- A lake is files in object storage: cheap and flexible, no guarantees; left alone it becomes a swamp.
- A lakehouse keeps the cheap files and adds an open table format (Delta) for ACID and versions, an engine (Spark) for SQL, and governance (Unity Catalog).

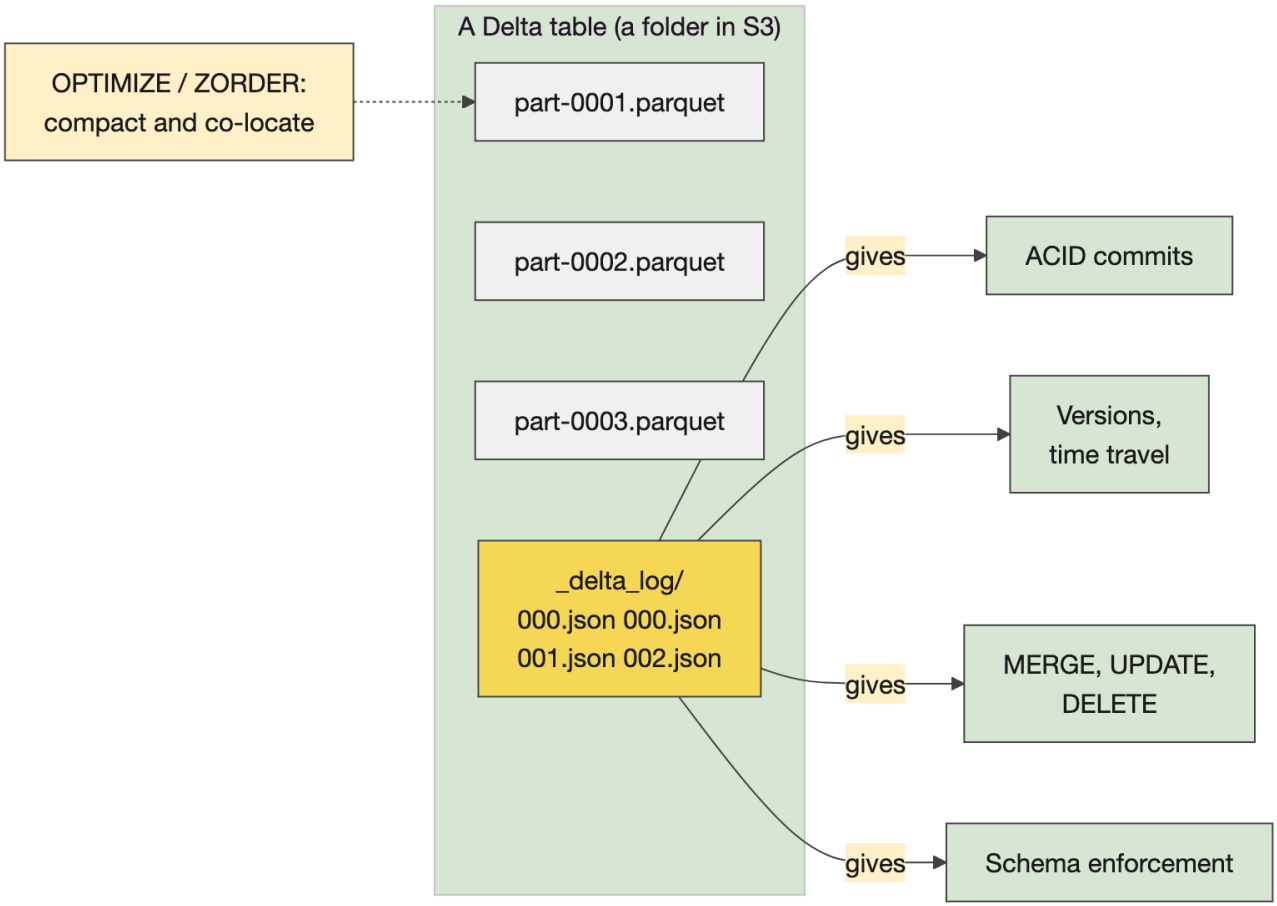
🖉 Whiteboard — warehouse, lake, lakehouse

- What does a warehouse give you, and what does it cost?
- What is a data lake?
- What goes wrong with lakes?
- What does a lakehouse add?


Sources
Vendor files
Databases
Streams, APIs
Ingestion
Auto Loader / COPY INTO
dbt seed, notebooks
Kafka, Lakeflow
Governance
Unity Catalog
catalog.schema.table
permissions, lineage
Storage: S3
Delta tables
Parquet + _delta_log
Compute: Databricks
Spark clusters



- Loading from different sources: files dropped in S3 (Auto Loader, COPY INTO), databases, streams and APIs (Kafka, Lakeflow), and in training dbt seed.
- Storage is your S3 bucket in Delta format; Databricks does not keep your data inside itself.
- Compute is Spark: clusters for notebooks and jobs, SQL warehouses (Photon) for dbt and BI. Serverless by default on the paved path.
- Unity Catalog names everything catalog.schema.table, holds permissions and lineage, and is what the Marketplace and Catalyst search. CG's three catalog types (application, source-aligned, domain) are the alignment layers made physical.



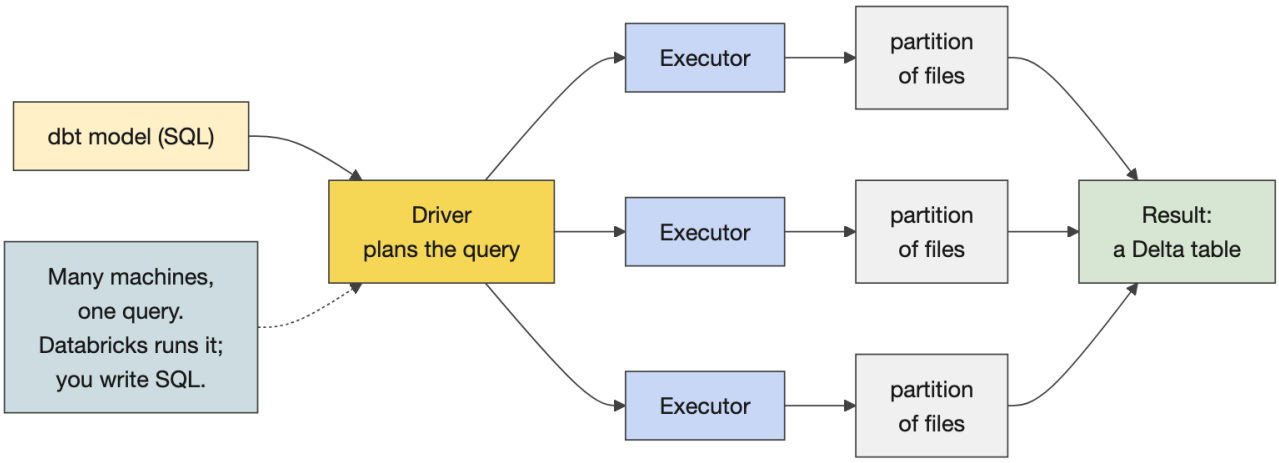
- The log is the trick: every write is a commit that lists which files make up the table now.
- That gives ACID, versions and time travel, MERGE and DELETE on files, and schema enforcement.
- OPTIMIZE compacts small files and ZORDER co-locates rows by the columns queries filter on; that is the post-hook in the learners' project.

> 🖥️ **DEMO — Delta under the hood, on this laptop**
>
> A Delta table is Parquet files plus a log: the cell writes one and reads every version back.

1. Run the cell: it writes the vendor prices as a Delta table, appends a day, merges a correction, reads version 0, compacts, and reads the table from DuckDB.
2. Open the table folder the script names: the Parquet files and `_delta_log/*.json`.

```
!python demos/delta-under-the-hood/demo.py
```

```
!ls -R demos/delta-under-the-hood/warehouse/prices_delta | head -20
!python -c "import json,glob; [print(json.dumps(json.loads(l))[:120]) for l in
open(sorted(glob.glob("demos/delta-under-the-
hood/warehouse/prices_delta/_delta_log/*.json"))[0]).read().splitlines()[:3]]"
# The log is readable JSON: a commit lists the files it added or removed.
```



- Spark splits data into partitions and runs the same work on each in parallel across machines; a shuffle (Exchange) regroups rows between stages.
- You write SQL; the driver builds a plan; Databricks provides the machines, the scheduling and the Delta integration.
- dbt never runs your data through its own process: it sends SQL to Databricks and Spark does the work next to the data.

> 🖥️ **DEMO — the Spark engine, locally (only if ahead of time)**
>
> A local Spark shows the driver, executors, partitions and the shuffle that Databricks runs for you at scale.

1. Run the cell (about thirty seconds): a local Spark with four executors loads the vendor prices multiplied up and runs a SQL aggregate.
2. Find *Exchange* in the printed physical plan: that is the shuffle between stages.
3. Run it again with `--hold 90` and open http://localhost:4040: Jobs, Stages, the SQL tab.

```
!python demos/spark-engine/demo.py 2>&1 | grep -vE 'WARN|^Setting|^To adjust'
```

WHAT, WHY AND HOW

How do all the tools fit together?

WHAT

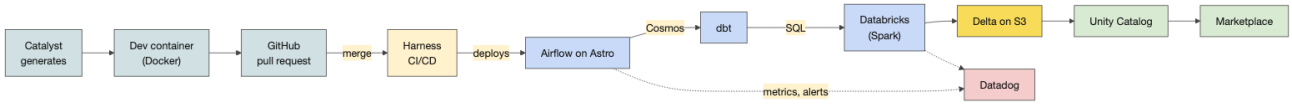
Catalyst generates; the dev container runs dbt and Airflow locally; GitHub holds the code; Harness deploys on merge; Airflow on Astro schedules; Cosmos turns the dbt project into tasks; dbt sends SQL to Databricks; Spark writes Delta on S3; Unity Catalog governs; the Marketplace publishes; Datadog watches.

WHY

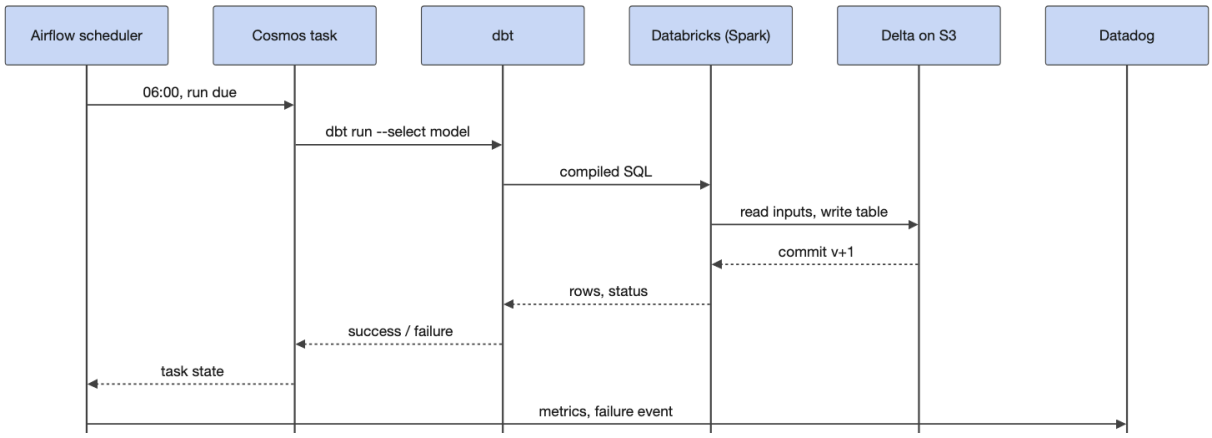
Every day this week zooms into one or two boxes; the picture is what connects them.

HOW

Elicit the role of each tool, then the diagram, then one scheduled run as a sequence.



- Left to right is the life of a change: generated with Catalyst, developed in the dev container, merged on GitHub, deployed by Harness.
- Top to bottom at run time: Airflow decides when, Cosmos maps the dbt project to tasks, dbt sends SQL, Databricks runs Spark, Delta is written to S3.
- Unity Catalog and the Marketplace are where consumers find the product; Datadog is where operations sees it run.



- At 06:00 the scheduler decides a run is due; each Cosmos task runs one dbt model; dbt sends compiled SQL; Spark reads inputs and commits a new Delta version.
- Status flows back up; a failure is an event in Datadog; the publish step is what consumers see.

🖉 Whiteboard — the paved path, tool by tool

- Which technologies are on the paved path?
- What does dbt provide? Databricks? S3?
- Airflow? Astronomer? Cosmos?
- Docker? Git? Harness? Datadog?
- When may you leave the path?

Shareback — one minute per table

> 💭 **SHAREBACK — one minute per table**
>
> Each table answers in one minute.

- One thing Databricks does that a plain file store cannot.
- One thing dbt relies on Databricks for.
- One tool on the diagram you have used, and one you have not.

4. Docker

Demo — Docker in the six steps of the challenge

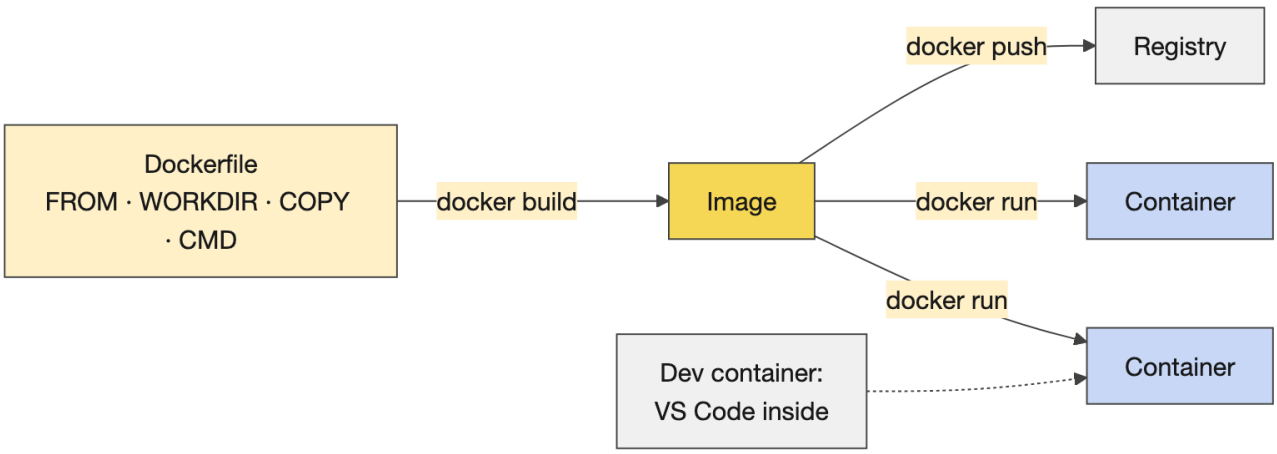
WHAT, WHY AND HOW

Why is your VS Code terminal not your laptop's terminal this week?

WHAT
A container bundles an application with its dependencies down to the OS user space; an image is the frozen template; a Dockerfile is the recipe.

WHY
Repeatable environments, isolation (network, ports, resources), the same bytes on a laptop, a build server and the cloud.

HOW
Run an image, look inside, write a four-line Dockerfile, build, run, tag; then the dev container's Dockerfile.



- FROM extends a base image: almost every Dockerfile starts that way; the training container extends an EDO image.
- An image is layered and cached: change the app and only COPY onwards rebuilds.
- A container lives as long as its command; a fresh container starts clean from the image.
- Isolation is the production point: one app per container, controlled ports, capped CPU and memory, a limited blast radius.

🖥️ DEMO — Docker in the six steps of the challenge

The same six steps, commands and outputs as `challenges/v1/01-docker-intro`, run here on the challenge's own six-holding app; you type them yourselves in the practical.

1. Check Docker Desktop is running: the whale in the menu bar, or `docker info`.
2. Step 1, run an image: `docker run --rm python:3.11-slim python --version`; the first run pulls the image, the container prints one line and is gone.
3. Step 2, look inside: `docker run --rm -it python:3.11-slim bash`, then `ls /` and `cat /etc/os-release` (Debian, not your laptop); a file written in `/tmp` is gone in the next container.
4. Step 3, the Dockerfile: four instructions in order, `FROM`, `WORKDIR`, `COPY`, `CMD`; the sample below is commented line by line; the starter leaves three of them as TODOs for you.
5. Step 4, build: `docker build -t portfolio-summary .` from the folder with the Dockerfile; the `.` is the build context; build twice and read `CACHED`; `docker images | grep portfolio-summary`.
6. Step 5, run: `docker run --rm portfolio-summary` prints exactly two lines; mount a different CSV with `-v` and the same image gives a different answer.
7. Step 6, tag: `docker tag portfolio-summary portfolio-summary:v1`; two names, one image ID; that is how CG ships Airflow and dbt images.
8. The gate: `mise run challenge:verify 01` from anywhere in the repository builds the starter image, runs it, checks the two lines and prints PASS.

Step	What you do	The command	What you should see
1 Run an image	Run a public image's Python	`docker run --rm python:3.11-slim python --version`	One line, `Python 3.11.x`; the container has already exited
2 Look inside	Open a shell in a container	`docker run --rm -it python:3.11-slim bash`, then `ls /`, `cat /etc/os-release`, `exit`	A Linux root listing; `os-release` names Debian
3 Write the Dockerfile	Fill the three TODO lines in `starter/Dockerfile`	`grep -E '^(FROM|WORKDIR|COPY|CMD)' Dockerfile`	Exactly four lines, in that order
4 Build	Bake the recipe into an image	`docker build -t portfolio-summary .` (twice)	Steps the first time; `CACHED` the second; `docker images` shows the row, about 130 MB
5 Run	Start a container from your image	`docker run --rm portfolio-summary`	`Portfolio summary` then `6 holdings, total market value 12,011,626.50`
6 Tag (optional)	Give the image a version	`docker tag portfolio-summary portfolio-summary:v1`	Two rows, `latest` and `v1`, one image ID
Gate	Prove it	`mise run challenge:verify 01`	`PASS`

If it stalls. `Cannot connect to the Docker daemon`: the engine is not running, start Docker Desktop. `permission denied` on the socket: use the dev container. `exec format error` or script not found: CRLF line endings after editing on Windows; save as LF and rebuild. A slow first `run` or `build`: the base image is being pulled, once.

```
!docker run --rm python:3.11-slim python --version
# Step 1. Python from inside a container built from an image, not from this laptop.
The container exited when the command did.
```

```
!docker run --rm python:3.11-slim bash -c "ls / && cat /etc/os-release | head -2"
# Step 2, non-interactive form (the challenge uses -it for a real shell). A Linux
root and a Debian os-release: the container's own filesystem.
```

```
!docker run --rm python:3.11-slim bash -c "touch /tmp/proof.txt && ls /tmp"
!docker run --rm python:3.11-slim ls /tmp
# Step 2, the variation: the file written in the first container is gone in the
second. Each container starts clean from the image.
```

```
!cat demos/docker/portfolio-summary/Dockerfile
!echo "--- the four instructions the check greps for:" && grep -E
"^(FROM|WORKDIR|COPY|CMD)" demos/docker/portfolio-summary/Dockerfile
# Step 3. A sample Dockerfile: base, then where, then what, then how to run. The
starter gives FROM and leaves the other three as TODOs.
```

```
!cd demos/docker/portfolio-summary && docker build -t portfolio-summary . 2>&1 | tail
-3
!cd demos/docker/portfolio-summary && echo "--- second build, CACHED lines:" &&
docker build -t portfolio-summary . 2>&1 | grep -cE 'CACHED'
!docker images | grep portfolio-summary
# Step 4. Build twice: the second build is almost entirely cached. The image row is
mostly the shared Python base layer.
```

```
!docker run --rm portfolio-summary
!echo "--- same image, a mounted CSV:" && docker run --rm -v
"$PWD/demos/docker/portfolio-summary/app/holdings.csv:/app/holdings.csv:ro"
portfolio-summary
# Step 5. Exactly the two lines the gate checks. Mount a different CSV and the same
image gives a different answer: image and data are separate.
```

```
!docker tag portfolio-summary portfolio-summary:v1 && docker images portfolio-summary
# Step 6. Two names, one image ID. Tag, push to a registry, pull elsewhere: that is
how CG ships Airflow and dbt images.
```

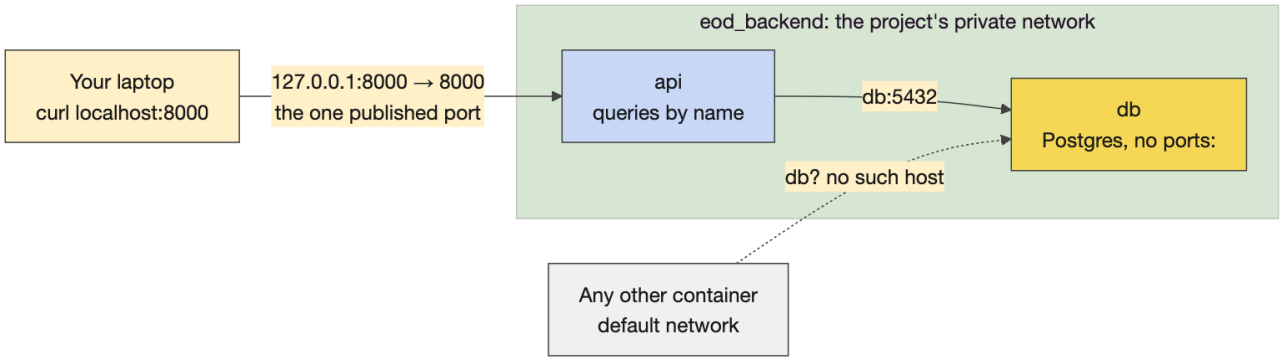
Demo — the dev container's Dockerfile

> **DEMO — the dev container's Dockerfile**
>
> The learners' dev container is a Dockerfile they can now read.

1. Open `reference/DataCreators-client/dev-container/edo-training-data-products/.devcontainer/Dockerfile`, the file VS Code builds for learners (the cell below prints its head).
2. Open Docker Desktop: Images, then Settings and Resources, briefly.

```
!sed -n "1,12p" ../../reference/DataCreators-client/dev-container/edo-training-data-
products/.devcontainer/Dockerfile
```

Demo (optional) — two containers with Compose, and the network between them



- Compose is one file for several containers: the services, a private network and a volume, started in dependency order.
- On that network a service's name is its hostname: the API connects to `db`, never to an address; Docker's DNS answers.
- `ports:` is the one door in; a service without it cannot be reached from the laptop or the office network, only by its neighbours.
- `astro dev start` writes exactly this kind of file for Airflow: scheduler, API server, triggerer and Postgres.

> 🖥️ **DEMO — two containers with Compose, and the network between them**
>
> `demos/docker/compose/`: Postgres seeded with the running example's price sheet, and a small HTTP API that queries it by name. Every line of the Compose file is commented; the eight cells below are `demo.sh` in order.

1. Open `docker-compose.yml`: two services (`db`, `api`), one network (`backend`), one volume. Read the comments on `networks:`, `ports:` and `depends_on:`; say the password is a demo value and point at the secure example for `.env`.
2. `docker compose up -d --build`, then `docker compose ps -a`: db has `5432/tcp` with no arrow (published nowhere); api has `127.0.0.1:8000->8000`.
3. `docker compose logs api`: the API started and was told the database's name, nothing more.
4. `docker network inspect eod_backend`: the two containers and their private addresses.
5. Inside api, resolve `db` and read `resolv.conf`: the service name is the hostname, Docker's embedded DNS at 127.0.0.11 answers, no address is written anywhere.
6. `curl localhost:8000/health` and `/prices?instrument_id=US0378331005` from the laptop: through the one published port, the API answers by querying db over the network.
7. Isolation three ways: `docker port` shows api published and db not; a container on the default network cannot resolve `db`; one started with `--network eod_backend` can.
8. `docker compose down -v`: containers, network and volume gone; the images stay.

```
!cd demos/docker/compose && (docker compose down -v --remove-orphans >/dev/null 2>&1;
docker compose config --services)
!cat demos/docker/compose/docker-compose.yml
# Step 0. A clean slate, then the file: two services, one network, one volume, every
line commented. Read the comments on networks:, ports: and depends_on:.
```

```
!cd demos/docker/compose && docker compose up -d --build 2>&1 | tail -8
!cd demos/docker/compose && docker compose ps -a --format 'table
{{.Service}}\t{{.Status}}\t{{.Ports}}'
# Step 1. Build the API image, pull Postgres, create the network and the volume,
start in dependency order: db first, then api once db is healthy.
# db shows 5432/tcp with no arrow: exposed inside, published nowhere. api shows
127.0.0.1:8000->8000.
```

```
!cd demos/docker/compose && docker compose logs --no-log-prefix api
# Step 2. The API's log: it started, and was told the database's name and nothing
more. No address anywhere: the network does the rest.
```

```
!docker network ls --filter name=eod_
!docker network inspect eod_backend --format '{{range .Containers}}{{println .Name
.IPv4Address}}{{end}}'
# Step 3. The network Compose created for the project, and who is on it: private
addresses nothing else on the laptop can route to.
```

```
!cd demos/docker/compose && docker compose exec api python -c "import socket;
print('db resolves to', socket.gethostbyname('db'))"
!cd demos/docker/compose && docker compose exec api grep nameserver /etc/resolv.conf
# Step 4. Inside the API container the service name is a hostname. Docker's embedded
DNS (127.0.0.11) answers for every user-defined network; no address is written
anywhere in the project.
```

```
!curl -s localhost:8000/health
!curl -s 'localhost:8000/prices?instrument_id=US0378331005'
# Step 5. From the laptop, through the one published port: the API answers by
querying db over the private network. The health route prints the name it was given
and the address DNS gave it.
```

```
!echo "api:" && docker port eod-api-1 && echo "db:" && docker port eod-db-1 && echo "
(nothing: db publishes no port, so no laptop address leads to it)"
!docker run --rm postgres:16-alpine pg_isready -h db -t 2 || echo "^ a container on
the default network cannot even resolve the name"
!docker run --rm --network eod_backend postgres:16-alpine pg_isready -h db -t 2
# Step 6. Isolation three ways: no laptop port leads to Postgres; a container outside
the network cannot find it; one joined to eod_backend can. Who talks to whom is a
line in the file.
```

```
!cd demos/docker/compose && docker compose down -v 2>&1 | grep -c Removed
!docker network ls --filter name=eod_
# Step 7. Down: the containers, the network and (with -v) the volume are gone; the
images stay, cached. On Day 4, astro dev start and astro dev stop do exactly this for
Airflow.
```

Challenge 7 — Docker practical

🎯 **ACTIVITY 7 — Docker practical · Individual or pairs · 65 min**

In the dev container, work through `challenges/v1/01-docker-intro/README.md` sections 1 to 5: run an image, look inside, write the Dockerfile, build the image, run your image. Start before lunch, finish after it. Section 6, the 'try a variation' steps and the stretch list are for anyone with time.

Produce:

- `docker run --rm portfolio-summary` printing the six-holding summary
- `mise run challenge:verify 01` showing PASS
- An answer to: why did the container stop, and what survives when you run a fresh one?

Finished early? Section 6 (tags); multi-stage builds; resource limits (`--memory`, `--cpus`); read the training repository's own Dockerfile and explain three lines of it; run the Compose demo in `demos/docker/compose/` and add a third, one-shot service that queries `db` by name and exits; or build and run the secure example in `common/glossaries/docker-secure-example/`.

Debrief questions:

- What is the difference between the image, the running process and its writable files?
- What travels with the image, and what does not?

🏆 **CHALLENGE TIME — Docker practical · before and after lunch, about 65 min**

Sections 1 to 5: run an image, look inside, write the Dockerfile, build it, run it. Lunch when you reach a good stopping point; back at 13:00. · `challenges/v1/01-docker-intro/README.md`

What learners do

1. `docker --version`; if the engine is not running, start Docker Desktop.
2. Section 1, run an image: a container from a public Python image.
3. Section 2, look inside: the container's own filesystem and processes.
4. Section 3, write the Dockerfile: `starter/Dockerfile` has one TODO per line.
5. Section 4, build the image.
6. Section 5, run the image: the app runs the same anywhere.

Gate. `mise run challenge:verify 01` prints PASS.

Support.

- Every Docker command by what it does, the Dockerfile instructions, and how to build and run containers securely: `common/glossaries/docker-commands.md`.
- Engine not running means Docker Desktop is not started on the laptop.
- CRLF line endings on Windows cause `exec format error`.
- The first pull is slow once.

Finished early? Section 6 (tags); the variations; resource limits; the Compose demo in `demos/docker/compose/`; multi-stage builds; the secure example in `common/glossaries/docker-secure-example/`.

🍽 **LUNCH · 60 min · back at 13:00**

Questions go in the table at the end; the next part starts on time.

Challenge 7 continued — with the stretch

Everyone: sections 3 to 5 to PASS. **Finished?** In this order, as far as time allows:

1. Section 6, tags, and the "try a variation" steps.
2. Resource limits: run your image with `--memory 128m --cpus 0.5` and read `docker stats`.
3. Compose: `bash demos/docker/compose/demo.sh`, then read its `docker-compose.yml` and add a third service: a one-shot `psql` (the `postgres:16-alpine` image with a `command:`) that runs one query against `db` by name and exits, the shape of a dbt run or an Airflow task; `docker compose up -d` and read its log.
4. A multi-stage build: a builder stage that installs, a final stage that only runs.
5. The secure example in `common/glossaries/docker-secure-example/`: build it, run it read-only as a non-root user, then `docker compose up` with the database and call `/db`.
6. Read the dev container's own Dockerfile and explain three lines to your neighbour.

Shareback — image, container, what survives

> 💭 **SHAREBACK — one table per question**
>
> Show the PASS count, then one table each.

- What is the difference between the image, the running process and its writable files?
- What travels with the image, and what does not?
- Why did the container stop, and what survives when you run a fresh one?
- One thing from the stretch list that surprised you.

5. Git and GitHub (optional)

Demo — the loop on a scratch repository, then the pull request on screen

WHAT, WHY AND HOW

Why will tomorrow's dbt code live on a branch and reach production through a pull request?

WHAT

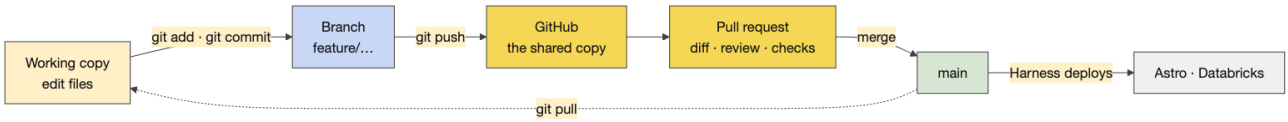
Git records a folder's history as commits on branches, on your machine. GitHub holds the shared copy and the pull request: the diff, the review, the checks and the merge.

WHY

The contract, the dbt models and the DAG are files. The pull request is where acceptance happens, Harness deploys what main contains, and Git remembers who changed what and why.

HOW

Six steps on a throwaway repository (init, status and diff, branch, merge, ignore, undo), the commands by objective, then the training repository's pull request on screen. The practical is the same loop on your Challenge 7 Dockerfile.



- A commit is a snapshot of the whole folder with an author, a time and a message; a branch is a named line of commits.
- Commit is local and push is what shares it: two verbs, two risks.
- A pull request is a conversation about a diff: the reviewer reads the changed lines, the checks run, and the merge is the decision.
- What main contains is what Harness deploys; nobody edits main directly.

> **DEMO — the loop on a scratch repository**
>
> Each cell is one step on a throwaway repository in `demos/git/sandbox`; the commands are the ones you type in the dev container.

1. Step 1, a repository and a first commit: `git init`, then `git add` and `git commit -m`; `git log --oneline` shows one commit with its message.
2. Step 2, what changed: edit the file; `git status --short` names it and `git diff` shows the lines. Read the diff before every commit: it is what the reviewer reads.
3. Step 3, a branch: `git switch -c feature/sttm-rules` and commit there; `git log --oneline --graph --all` shows the branch one commit ahead of main.
4. Step 4, the merge: back on main, `git merge --no-ff` brings the branch in; on GitHub that is the approved pull request's merge button.
5. Step 5, what never goes in: a `.env` with a token appears in `git status`; one line in `.gitignore` and Git no longer offers it.
6. Step 6, undo: `git restore` drops an uncommitted edit; `git revert` undoes a commit with a new commit; history on a shared branch is never rewritten.

```
!rm -rf demos/git/sandbox && mkdir -p demos/git/sandbox
!cd demos/git/sandbox && git init -q -b main && git config user.name 'Data Creator'
&& git config user.email 'creator@example.com'
!cd demos/git/sandbox && printf '# EOD prices: data contract\n\nOwner: pricing data
team\n' > contract.md && git add contract.md && git commit -q -m 'Add the EOD prices
contract skeleton'
!cd demos/git/sandbox && git log --oneline
# Step 1. A repository is a folder with a history. The first commit records the file,
who made it, when and why.
```

```
!cd demos/git/sandbox && printf '\n## Rules\n\n- price_gbp = price_local * fx_rate
(derive)\n' >> contract.md
!cd demos/git/sandbox && git status --short
!cd demos/git/sandbox && git diff
# Step 2. status names the changed files; diff shows the lines. Read the diff before
every commit: it is what the reviewer reads.
```

```
!cd demos/git/sandbox && (git switch -c feature/sttm-rules 2>/dev/null || git switch
feature/sttm-rules)
!cd demos/git/sandbox && git add contract.md && git commit -q -m 'Add the price_gbp
derive rule to the STTM'
!cd demos/git/sandbox && git log --oneline --graph --all
# Step 3. Work on a branch, never on main. The branch carries your commits until a
pull request brings them back.
```

```
!cd demos/git/sandbox && git switch -q main && git merge --no-ff -q -m 'Merge
feature/sttm-rules: the price_gbp rule' feature/sttm-rules
!cd demos/git/sandbox && git log --oneline --graph
!cd demos/git/sandbox && cat contract.md
# Step 4. What GitHub does when the pull request is approved: main now has the rule,
and the history shows where it came from.
```

```
!cd demos/git/sandbox && printf 'DATABRICKS_TOKEN=dapi-not-a-real-token\n' > .env &&
git status --short
!cd demos/git/sandbox && printf '.env\n' > .gitignore && git add .gitignore && git
commit -q -m 'Ignore the local secrets file' && echo '--- after .gitignore:' && git
status --short
# Step 5. Secrets and generated files stay out of history: list them in .gitignore
before the first commit.
```

```
!cd demos/git/sandbox && printf 'oops\n' >> contract.md && git status --short && git
restore contract.md && echo '--- after restore:' && git status --short
!cd demos/git/sandbox && printf 'oops\n' >> contract.md && git commit -qam 'A
mistake' && git revert --no-edit HEAD >/dev/null && git log --oneline -3 && tail -1
contract.md
# Step 6. restore drops an uncommitted edit; revert undoes a commit with a new one.
History on a shared branch is never rewritten.
```

Objective	Command	What you see
Get the shared copy	`git clone <url>`	A folder with its whole history inside
Where am I, what changed	`git status`, `git branch --show-current`	The changed files; the branch name
The exact lines	`git diff`, `git diff --staged`	Lines removed and added, file by file
Record a change	`git add <file>`, `git commit -m "<what and why>"`	A new commit on the current branch
Read the history	`git log --oneline --graph --all`	One line per commit, branches drawn
Work on a branch	`git switch -c <name>`, `git switch main`	A new branch from here; back to main
Share, catch up	`git push -u origin <branch>`, `git pull`	Your commits on GitHub; others' commits here
Undo	`git restore <file>`, `git restore --staged <file>`, `git revert <commit>`	An edit dropped; a file unstaged; a commit undone by a new one
Park an edit	`git stash`, `git stash pop`	Changes set aside, then brought back
Keep things out	`.gitignore`	`.env`, `target/`, data files: never offered

In VS Code the Source Control view runs the same commands behind buttons: stage, commit, branch, push, and the diff on every changed file. The full list, by objective, with the files Git reads and the safe-working habits: `common/glossaries/git-commands.md`.

> 🖥️ **DEMO — the pull request on screen**
>
> The training repository in CG's GitHub organisation, from the dev container: the practical's steps, done once.

1. `git remote -v` and `git status`: the shared copy's address, and the Dockerfile Challenge 7 changed.
2. `git switch -c feature/<username>-docker`, `git add challenges/v1/01-docker-intro/starter/Dockerfile`, `git commit -m 'Challenge 7: the portfolio-summary Dockerfile'`.
3. `git push -u origin feature/<username>-docker`: GitHub prints the pull-request link.
4. The pull request page: a title, a description (what, why, how to verify: `mise run challenge:verify 01`), the Files changed tab with the four lines, one review comment, the checks, the merge button (not pressed).

Optional practical — branch, commit, push, pull request

🏆 **CHALLENGE TIME — Your Dockerfile in a pull request · 15 min, or homework**

The loop from the demo, on the file you wrote in Challenge 7, in the dev container. · `challenges/v1/01-docker-intro/starter/Dockerfile`

What learners do

1. `git status`: the Dockerfile you edited is listed as modified, and nothing else.
2. `git switch -c feature/<your-username>-docker`: a branch of your own.
3. `git add challenges/v1/01-docker-intro/starter/Dockerfile`, then `git commit -m '<what and why>'`: one commit.
4. `git log --oneline --graph -3`: your commit on your branch, ahead of main.
5. `git push -u origin feature/<your-username>-docker`, open the link GitHub prints, create the pull request: a title and three lines (what, why, how to verify).
6. Ask your neighbour to review it: one comment on the diff. Do not merge.

Gate. `git log --oneline --graph -3` shows your commit on your branch; the pull request exists, with a description and one review comment.

Support.

- No push rights to the training repository: stop after the log; the pull request steps were the demo.
- `Please tell me who you are`: set `git config --global user.name` and `user.email` once.
- An SSL certificate error on push: the dev container's `TROUBLESHOOTING.md`, under Git SSL certificate errors.
- Every Git and GitHub command by objective, the files Git reads, and how to work safely with branches, pull requests and credentials: `common/glossaries/git-commands.md`.
- The commands by objective are in the table above; VS Code's Source Control view does the same with buttons.

Finished early? A second commit on the branch, and watch the pull request update. `git revert` the first commit and read the diff. Read the training repository's `pre-commit-config.yaml` and say what each hook checks before a commit is allowed.

☕ **BREAK · 15 min · back at 14:15**

Questions go in the table at the end; the next part starts on time.

6. Catalyst skills, and setting up for tomorrow

Demo — the skills, their tiers and the lifecycle; how to load them

WHAT, WHY AND HOW

Which skill do I reach for, and when?

WHAT

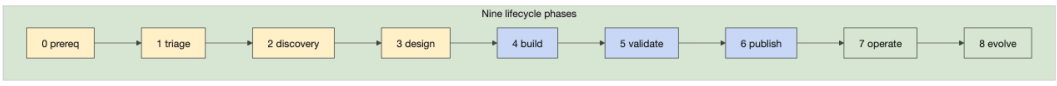
About a hundred Catalyst skills in four tiers that compose downwards, each tagged with the lifecycle phase it serves; the five you have used sit in discovery and design.

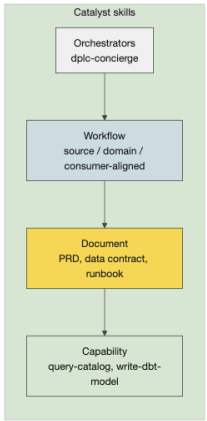
WHY

Knowing the map means asking Catalyst for the right document at the right phase, instead of generating code before the design is agreed.

HOW

The tiers, the phase table, how the plugin loads, then (if time) one design skill on your own product. No build skills today.





- Orchestrators run a phase; workflow skills run several steps; document skills make one artefact from a template; capability skills do one action.
- A skill is a Markdown instruction file with a template and checks, sometimes with Python; Copilot shows it reading the file.

Tier	How many	What they are	Examples
Orchestrators	11	Run a lifecycle phase or stage end to end	`dplc-concierge`, `dplc-triage`, `dplc-design`, `dplc-build`, `dplc-publish`
Workflow	15	Several steps, one job	`source-aligned`, `domain-aligned`, `consumer-aligned`, `setup-dbt-project`, `data-modeling`
Document	37	One artefact from a template	`product-requirements-document`, `data-contract`, `transformation-spec`, `sla-dashboard-spec`, `runbook`
Capability	42	One action, composable	`connect-databricks`, `query-table`, `profile-columns`, `write-dbt-model`, `generate-dag`, `validate-dag`

Phase	The document skills you would ask for	This week
1 Triage	`marketplace-search`, `consumer-use-case-map`, `demand-tracker-entry`	Day 1 (does it exist? who consumes it?)
2 Discovery	`product-requirements-document`, `feasibility-report`, `business-case`, `effort-estimate`, `requirements-acceptance-criteria`	Day 1
3 Design	`data-contract`, `transformation-spec`, `data-model`, `architecture-diagram`, `data-quality-controls-plan`, `nfr-specification`, `security-design`	Day 1 and today
4 Build	`setup-dbt-project`, `write-dbt-source`, `write-dbt-model`, `validate-dbt`, `adding-dbt-unit-test`, `generate-dag`, `validate-dag`	Wednesday and Thursday
5 Validate	`data-quality-sign-off`, `uat-sign-off`	Friday's evidence
6 Publish	`release-notes`, `consumer-onboarding-guide`, `deployment-record`	Thursday's Phase 2
7 Operate	`runbook`, `sla-dashboard-spec`, `incident-log`	Today (service levels) and Thursday
8 Evolve	`downstream-impact-assessment`, `deprecation-notice`, `usage-analytics-report`	Mentioned

🖥️ DEMO — how the skills load, and how you know one ran

The plugin is three settings and one click; a skill is a file you can read.

1. VS Code settings: `chat.plugins.enabled`, `chat.useAgentSkills`, and `chat.plugins.marketplaces` pointing at the Catalyst repository (or a local copy of it).
2. Reload; in Copilot Chat run `/plugins` and enable AI Catalyst; `/ai-catalyst` then lists the skills.
3. Open one skill's `SKILL.md` and its template: the instructions, the sections, the checks.
4. Run `/ai-catalyst data-contract` on the running example and point at the moment Copilot reads the skill file; without that line it is plain Copilot.
5. Same skills in Claude Code on this laptop, as `/data-contract`, from `courseware/.claude/skills/`.

Activity 8 — one design skill on your product (if time allows)

🎯 ACTIVITY 8 — Try one design skill on your product (if time allows) · Pairs · 20 min

Pick one design-phase skill and run it on your own PRD and contract: `consumer-use-case-map` (who consumes it and how), `data-model` (the entities and their keys), `architecture-diagram` (the flow from sources to consumers), `data-quality-controls-plan` (the few checks that matter) or `nfr-specification` (performance, reliability, cost). Attach the PRD and the contract. Read the result for the invented detail before you keep anything. No build skills today: dbt comes tomorrow, with the design agreed.

Produce:
• One generated design document, reviewed
• One line for the room: what it added that the PRD and contract did not say, and one thing it invented

Finished early? Run a second skill and check whether the two documents agree with each other.

Debrief questions:
• Which phase was the skill for, and did your product have what that phase needs?
• What did it invent, and which input would have prevented it?

🤖 **PROMPT**

LEARNERS Copilot Chat · Chat view, PRD and contract attached

```
/ai-catalyst consumer-use-case-map
Map the consumers, their use cases, access patterns and SLA needs for
the product in the attached PRD and contract. Mark anything you could
not find in them as an open question.
```

INSTRUCTOR Claude Code

```
/consumer-use-case-map with @courseware/common/running-example/02-
prd.md and @courseware/common/running-example/03-data-contract.md
```

Databricks setup — before you leave

🖥️ **DEMO — where tomorrow's tables will land**

Everyone sets up Databricks now, because tomorrow's first command is dbt debug.

1. In the dev container: `mise run setup:databricks` (browser OAuth; press Enter through the prompts; it takes minutes per person).
2. Then `dpc-check`: every gate green.
3. Databricks UI: Catalog, `training_dev`, `training_‹username›`; tomorrow every dbt table appears there.
4. Open `challenges/v1/02-dbt-pipeline/README.md` tonight if you want a head start; the first hour tomorrow is sources and staging.

Close

✅ **BEFORE LEARNERS LEAVE**

□ Each table's contract extended with mapping rules and measurable service levels, shared for review.
□ Challenge 7 at PASS for everyone (paired where an engine was blocked); the stretch noted for those who took it.
□ Databricks setup done for everyone: dpc-check green, the training schema visible, because tomorrow's first command is dbt debug.
□ Access problems recorded separately from content questions, with an owner for each; a platform-support contact for tonight.

Questions from the room

Type the question in the table as it comes; answer live in the cells below; tidy up at the break. Anything unanswered goes to the platform team or the Teams channel, with the person who asked.

#	Question	Who asked	Answer / where it went
1			
2			
3			

```
# Answer live here: SQL against a demo warehouse, a small Python check, or a shell command.
# Example: sql("select * from training_demo.stg_holdings limit 5", HERE / "demos/portfolio_analytics/warehouse/training_dev.duckdb")
```

```
# Sketch a diagram live (before Excalidraw). Edit and run.
quick_diagram("""
flowchart LR
  A[source file] --> B[dbt rules on Databricks] --> C[consumer table]
""")
```

Answers written up after the day — fill in here, then share the notebook.

-

© 2026 Decoded Limited. All rights reserved. decoded.com